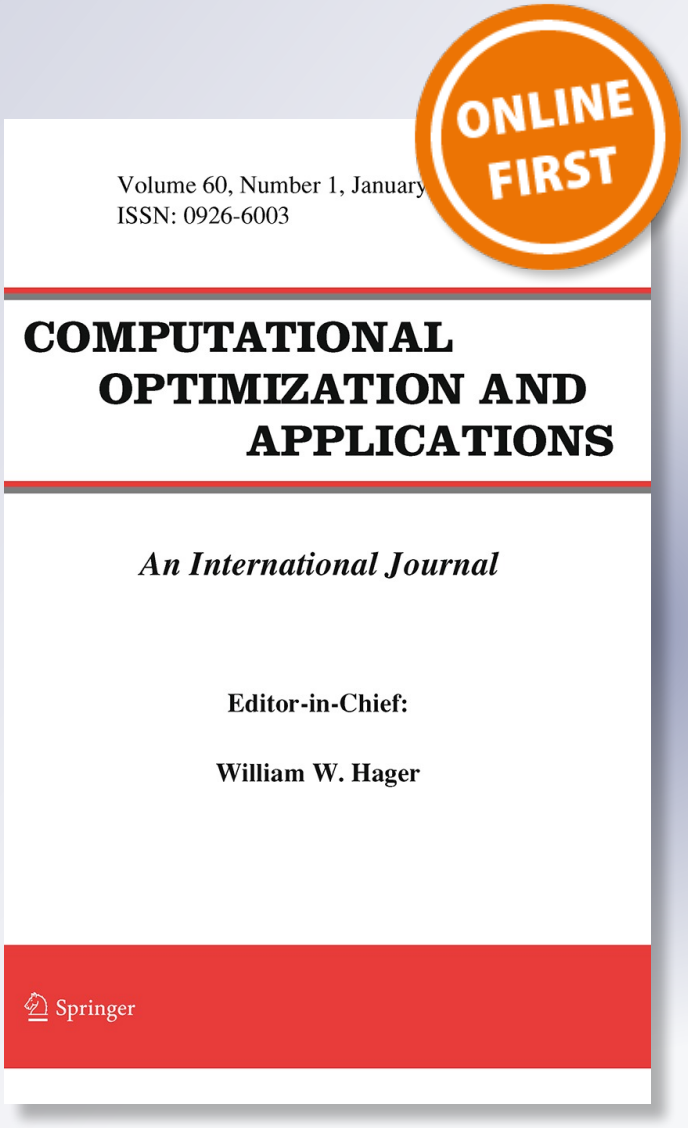
An exact algorithm for the sequential ordering problem and its application to switching energy minimization in compilers

Ghassan Shobaki & Jafar Jamal

Computational Optimization and Applications
An International Journal

ISSN 0926-6003

Comput Optim Appl
DOI 10.1007/s10589-015-9725-9



Springer

Your article is protected by copyright and all rights are held exclusively by Springer Science +Business Media New York. This e-offprint is for personal use only and shall not be self-archived in electronic repositories. If you wish to self-archive your article, please use the accepted manuscript version for posting on your own website. You may further deposit the accepted manuscript version in any repository, provided it is only made publicly available 12 months after official publication or later and provided acknowledgement is given to the original source of publication and a link is inserted to the published article on Springer's website. The link must be accompanied by the following text: "The final publication is available at link.springer.com".



Author's personal copy

Comput Optim Appl
DOI 10.1007/s10589-015-9725-9

An exact algorithm for the sequential ordering problem and its application to switching energy minimization in compilers

Ghassan Shobaki · Jafar Jamal

Received: 5 August 2014
© Springer Science+Business Media New York 2015

Abstract This article presents an exact algorithm for the precedence-constrained traveling salesman problem, which is also known as the sequential ordering problem. This NP-hard problem has applications in various domains, including operational research and compilers. In this article, the problem is presented and solved in the context of minimizing switching energy in compilers. Most previous work on minimizing switching energy in the compiler domain has been limited to simple heuristics that are not guaranteed to give an optimal solution. In this work, we present an exact algorithm for solving the switching energy minimization problem using a branch-and-bound approach. The proposed algorithm is simple and intuitive, yet powerful. It is the first exact algorithm for the switching energy problem that is shown to solve real instances of the problem within a few seconds per instance. Compared to previous work in the operational research domain, the proposed algorithm is believed to be the most powerful exact algorithm that does not require a linear programming formulation. The proposed algorithm is experimentally evaluated using instances taken from a production compiler. The results show that with a time limit of 10 ms per node, the proposed algorithm optimally solves 99.8 % of the instances. It optimally solves instances with up to 598 nodes within a few seconds. The resulting switching cost is 16 % less than that produced without energy awareness and 5 % less than that produced by a commonly used heuristic.

Keywords Compiler optimizations · Sequential ordering problem · Precedence-constrained traveling salesman problem · Instruction scheduling · Branch-and-bound enumeration · NP-complete problems · Low power computing · Low energy computing · Switching energy

G. Shobaki (✉) · J. Jamal
Princess Sumaya University for Technology, Amman, Jordan
e-mail: g.shobaki@psut.edu.jo

Published online: 25 February 2015

Author's personal copy

1 Introduction

The *precedence-constrained traveling salesman problem (PCTSP),* which is also known in the literature as the *sequential ordering problem (SOP),* is an NP-hard problem that has applications in various domains, including operational research and compilers. The problem may be simply described as follows. Given a weighted graph and a dependence graph representing precedence constraints among the nodes, find a minimum-cost *Hamiltonian path* in the weighted graph that satisfies the precedence constraints in the dependence graph. A Hamiltonian path is a simple path that visits every node in the graph exactly once, and the cost of the Hamiltonian path is the sum of edge weights on the path. Since the SOP is more general than the traveling salesman problem (TSP), which is a well-known NP-hard problem [7], the SOP is necessarily NP-hard.

The SOP has been studied in operational research, because it has many interesting applications in manufacturing, production planning and transportation [9,16]. Various algorithms have been proposed for solving this problem in the operational research domain, including exact algorithms by Ascheuer et al. [2] and Montemanni et al. [16] and a precise heuristic by Gambardella et al. [15].

The SOP also has an interesting application in compilers. A compiler translates a program written in a high-level programming language, such as C and C++, into machine instructions. After generating machine instructions, the compiler applies a number of optimizations to the generated instructions. One of these optimizations is *instruction scheduling*, in which the compiler tries to find the best order for executing the instructions. Traditionally, the objective of instruction scheduling is finding an instruction order that gives the minimum execution time. However, since reducing energy consumption has recently become an important issue, some researchers have studied instruction scheduling with the objective of minimizing the switching energy [18,25]. In this formulation of the scheduling problem, the switching energy between two consecutive instructions is modeled by the *Hamming distance* between the binary encodings of the two instructions. The Hamming distance between two equal-length bit strings is the number of positions in which the two corresponding bits are different. For example, given two instructions with encodings 0011 and 1000, the Hamming distance for this pair of instructions is 3, because the two encodings have different bit values in 3 out of 4 positions. This model is based on the fact that in the actual hardware implementation, each bit value corresponds to a certain voltage level (for example, high voltage represents a One and low voltage represents a Zero). When an instruction is fetched from memory to the CPU, differences in bit values between the current instruction and the previous instruction require switching the voltage from high to low or vice versa. This voltage switching consumes energy. Therefore, minimizing the encoding differences between two consecutive instructions will minimize the switching energy associated with the execution of this instruction pair. Given an instruction sequence, the total switching energy of the sequence is minimized by minimizing the sum of switching energy values across all instruction pairs in the sequence.

Given this cost model, the problem of scheduling for minimum switching energy may be formulated as an SOP as follows [4]. Each instruction is represented by a node in a weighted graph, in which each edge weight is the Hamming distance for the

Author's personal copy

corresponding instruction pair. A dependence graph is built to capture the precedence constraints among instructions to ensure correct execution. For example, if Instruction y uses the result produced by Instruction x, x must appear before y in any correct schedule (y is dependent on x). This precedence constraint is represented by an edge from x to y in the dependence graph to ensure that x is always scheduled before y. Given these two graphs, the problem of finding a correct instruction order (schedule) that minimizes the total switching energy is equivalent to the problem of finding a minimum-cost Hamiltonian path in the weighted graph that satisfies the precedence constraints in the dependence graph, which is a SOP.

Most previously proposed algorithms for the switching energy minimization problem in compilers have been based on simple, relatively imprecise, heuristics [18,25]. Apparently researchers who have worked on this problem in the compiler domain were not aware of the work that has been done on the equivalent SOP in the operational research domain.

In this article, we present a branch-and-bound algorithm for computing exact solutions to the switching energy minimization problem. We have implemented the algorithm and integrated it into the LLVM open-source Compiler. We have also implemented and integrated the simple *nearest neighbor* (NN) heuristic proposed in previous work [18,25]. We have used each of these algorithms to schedule instructions targeting an ARM processor for a large subset of the MiBench benchmarks [10]. The performance of the different scheduling algorithms was then evaluated based on the switching cost of the resulting instruction streams. The results of this experimental evaluation show that with a time limit of 10 ms per instruction, the proposed branch-and-bound algorithm optimally solves 99.8 % of the instances. It optimally solves instances with up to 598 instructions. The resulting schedules have an overall switching cost that is 16 % less than that of the schedules produced without energy awareness and 5 % less than that of the schedules produced by the NN heuristic.

The contributions of this article may be summarized as:

1. Formulating the problem of switching energy minimization in compilers as a SOP, thus linking the work that has been done on switching energy minimization in compilers to the work that has been done on the SOP in operational research. The ideas and techniques presented in the current article may be applied equally well in both domains.
2. Presenting a new exact algorithm for solving the SOP, using a branch-and-bound approach. The proposed algorithm is based on a collection of simple and intuitive ideas, yet the experimental results show that it performs extremely well. Due to the comprehensive set of pruning techniques used in the algorithm, it is believed to be the most powerful exact SOP algorithm that does not require a mixed integer linear programming (MILP) formulation, thus eliminating the need for an MILP solver. Compared to existing algorithms in the compiler domain, the proposed algorithm is the first exact algorithm that is shown experimentally to solve the vast majority of the studied instances in a production compiler within reasonable time (a fraction of a second for most instances).
3. Integrating the proposed algorithm into the LLVM open-source compiler and evaluating its performance on real code targeting the ARM architecture, which is one of

Author's personal copy

the most commonly used architectures in embedded and mobile devices. The data set studied in this paper includes many large and challenging instances of the problem. The experimental evaluation involves a thorough study of the impact of each pruning technique and the interactions among the different pruning techniques.

The rest of the article is organized as follows. Section 2 defines the problem and states the assumptions. Section 3 summarizes related previous work. Section 4 describes the heuristic technique proposed in previous work and used in the experimental evaluation, and Sect. 5 describes the proposed branch-and-bound algorithm. Section 6 presents the experimental results, and Sect. 7 summarizes the article's contributions and outlines future work.

2 Poblem description

In this section we precisely define the problem addressed in this article and introduce the key terms and concepts. We also explicitly state all the assumptions that are being made.

A compiler determines the instruction order in the instruction scheduling pass. Typically, a compiler has two instruction scheduling passes: one before register allocation (*pre-allocation scheduling*) and one after register allocation (*post-allocation scheduling*). This work focuses on post-allocation scheduling, because the instruction encoding that determines the switching energy must include register encodings, which are not known until after register allocation.

As explained in the problem formulation, precedence constraints in compiler instruction scheduling represent instruction dependencies that must be satisfied to ensure correct execution. In post-allocation instruction scheduling, additional constraints are added to avoid conflicts on registers. For example, if Instruction x *produces* a certain value and stores it in a certain register, and some other instructions *consume* this value by reading this register, all consuming instructions must be scheduled before any instruction that stores a new value in this register (a new producer). This is ensured by adding precedence constraints from the first producer to the second producer (write-after-write dependency) and from every consumer of the first value to the second producer (write-after-read dependencies).

A compiler divides the code into straight-line pieces of code, called *basic blocks*. A basic block is a sequence of instructions with no branches, except possibly the last instruction in the sequence. Due to the complexity of processing a whole program (or even a whole procedure) at once, a compiler usually applies its optimizations to one *region* at a time. Depending on the optimization, the region may consist of one or more basic blocks. In this article, it is assumed that the region for instruction scheduling consists of a single basic block (*local instruction scheduling*), which is a valid assumption for many production compilers, including the LLVM compiler used in our experimental evaluation. It is also assumed that all instructions have equal-length encodings, which is a valid assumption for the ARM architecture, the target architecture in our experimental evaluation.

The problem addressed in this article is finding a post-allocation instruction schedule with minimum *switching cost* for a given basic block. The *switching cost* for an

Author's personal copy

instruction schedule is defined as the sum of the Hamming distances of all pairs of consecutive instructions in the schedule. The Hamming distance of a pair of consecutive instructions is the number of positions in which the two instruction encodings have different bit values. An instance of the switching energy problem is represented by a *cost graph* that encodes the Hamming distances and a *data dependence graph* (DDG) that encodes instruction dependencies. These graphs are structured as follows.

The DDG is a directed acyclic graph (DAG) in which each instruction is represented by a node and each dependency is represented by an edge. If Instruction y depends on Instruction x, the DDG will have an edge from Node x to Node y. Given two instructions x and y in the DDG, if there is a path from x to y in the DDG, it is said that x is a *predecessor* of y and y is a *successor* of x. In this case, Instructions x and y are said to be *dependent* instructions. If the path from x to y consists of a single edge, x is said to be an *immediate predecessor* of y, and y is said to be an *immediate successor* of x. If, on the other hand, there is no path in the DDG connecting Instructions x and y in either direction, Instructions x and y are said to be *independent* instructions. An instruction in the DDG that has no predecessors is called a *root instruction*, and an instruction that has no successors is called a *leaf instruction*.

The cost graph is a complete weighted graph in which each instruction is represented by a node and each Hamming distance is represented by an edge weight. For every pair (x, y) of instructions, the weight of the edge (x, y) is the Hamming distance of the instruction pair (x, y). Note that since the Hamming distance between two instructions is symmetric, the cost graph representing Hamming distances will be an undirected graph. It is important to emphasize though that the algorithm proposed in this article does *not* assume that the cost graph is undirected; it will work correctly if the cost graph is a directed graph encoding *asymmetric* edge weights. It is also noted that although Hamming distances satisfy the triangle inequality [24], the proposed algorithm does *not* assume that edge weights in the cost graph satisfy the triangle inequality.

Given a DDG and a cost graph, a *feasible schedule* is a permutation of the nodes that satisfies the dependence constraints in the DDG. In other words, a feasible schedule is a Hamiltonian path in the cost graph that satisfies the precedence constraints in the DDG. The switching energy minimization problem is the problem of finding a feasible schedule with minimum switching cost. This problem is equivalent to the SOP studied in operational research [2,9,15,16]. The problem is also equivalent to the PCTSP, if the PCTSP objective is defined as finding a minimum-cost Hamiltonian path rather than a Hamiltonian cycle.

It is noted that in the above definition of the switching energy problem, the start and end instructions are *not* fixed. It is the responsibility of the solver to find the start and end instructions that give the minimum total cost. The start instruction must be one of the root instructions and the end instruction must be one of the leaf instructions.

In the above problem definition, the scheduling objective is limited to switching energy minimization without accounting for schedule length minimization ((exploiting instruction-level parallelism (ILP)). This is based on the assumption that the target processor has an out-of-order execution engine that will dynamically reorder instructions to exploit ILP at run time. To a very good approximation, this is a valid assumption on many modern out-of-order processors for a wide range of programs [23]. The target processor in the experimental evaluation of the present article is a modern out-of-order

Author's personal copy

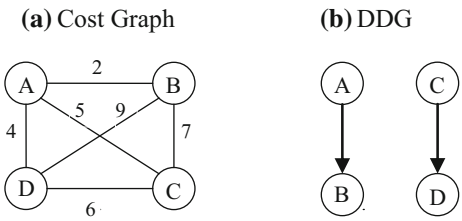


Fig. 1 An instance of the switching energy problem

ARM processor. It is also noted that out-of-order execution does not affect the switching energy, because switching takes place in the fetching stage, and instructions are always fetched from memory in the order determined by the compiler.

Figure 1 shows an instance of the switching energy problem with four instructions. Figure 1a shows the cost graph, in which each edge weight represents the Hamming distance for the corresponding instruction pair. For example, the Hamming distance between Instructions A and B is 2. Figure 1b shows the DDG, in which Instruction B depends on Instruction A and Instruction D depends on Instruction C. Therefore, Instruction A must appear before Instruction B and Instruction C must appear before Instruction D in any feasible schedule. Instructions A and C are root instructions and Instructions B and D are leaf instructions. Instructions A and C are, for example, two independent instructions that may be scheduled in any order. This instance is used throughout the article to illustrate the proposed algorithmic techniques.

3 Prior work

Compiling for low energy has been studied by many researchers [5,12,17,18,27,28]. The early work of Tiwari et al. [28] mentions several compiler optimizations that may reduce a program's energy consumption, including reordering instructions to minimize the switching energy (the topic of the current article), energy-aware instruction selection and improving register allocation to reduce memory accesses. The earliest work on scheduling for minimum switching energy is perhaps the *cold scheduling* algorithm proposed by Su et al. [25]. This algorithm is a greedy algorithm based on list scheduling, the most widely used scheduling technique in production compilers [6]. In cold scheduling, a list of ready instructions is maintained, and, at each point, the ready instruction with the lowest contribution to the total energy consumption is selected. Viewing the switching energy minimization problem as a PCTSP, the heuristic used in cold scheduling is equivalent to the Nearest Neighbor (NN) heuristic, which is known to give sub-optimal solutions in many cases [24]. The NN heuristic is described in Sect. 4. Interestingly, an experimental study by Charikar et al. [4] of several heuristics for the PCTSP shows that the PCTSP is very hard to approximate using heuristics, and that the NN heuristic performs relatively well in practice compared to other heuristics.

Tomiyama et al. [26] present a more precise algorithm for the switching energy problem. The algorithm is based on an enumerative search of the solution space. To avoid excessive solution times, however, they only consider a limited number of options at each point, thus speeding up the search at the price of losing optimality. Interestingly, they use a pruning idea that is similar to the suffix-schedule-reuse idea

Author's personal copy

described as part of the history utilization technique in Sect. 5.5. However, the history utilization technique described in the present article is far more general.

Lee et al. [14] prove that the switching energy minimization problem is NP-hard by a reduction from the Hamiltonian-Path problem, which is closely related to the TSP problem. Then, they describe a heuristic for solving this problem based on weighted bipartite matching.

Parikh et al. [18] explore multiple heuristic algorithms for balancing switching energy and schedule length. Their simulation-based results show that energy-oriented scheduling algorithms (using various derivatives of the NN heuristic) save energy by up to 30 % over performance-oriented scheduling. They also report that both performance-oriented and energy-oriented algorithms produce comparable results in terms of execution cycles (schedule length). The DDGs used in their experiments, however, were relatively small; the largest DDG had 64 instructions.

Shao et al. [19] describe a heuristic scheduling technique, named SAMLS (Switching Activity Minimization Loop Scheduling), which reduces both schedule length and switching energy in loops. The algorithm is based on improving an initial schedule by repeatedly rescheduling the instructions using rotation scheduling and bipartite matching.

Bednarski and Kessler [3] propose an algorithm for generating energy-optimal schedules using an enumerative search. Their experimental results, however, show that their algorithm is practical only for basic blocks with 30 instructions or less. For basic blocks with 40 instructions or more, the algorithm does not complete the search within the 6-hour time limit. In order to avoid long compile times, they use heuristics to skip certain parts of the solution space, thus trading optimality for faster compilation.

As mentioned in the previous section, the switching energy minimization problem is equivalent to the SOP that was introduced in 1988 as an operational research problem by Escudero, who described an inexact algorithm for solving it [9]. Previous work on the SOP in the operational research domain included more precise algorithms than those proposed in the compiler domain. Ascheuer et al. [2] present an MILP formulation of the problem and describe a branch-and-cut algorithm that computes exact solutions. They show experimentally that their branch-and-cut algorithm can optimally solve real-world instances with more than 200 nodes within a few minutes per instance.

A more recent exact algorithm is proposed by Montemanni et al. [16]. They describe a branch-and-bound algorithm that can be integrated in an MILP solver. The algorithm recursively decomposes a large problem into smaller sub-problems that can be solved independently and then recombines sub-problem solutions. The decomposition is based on identifying a *fixed vertex* in the DDG. According to the terminology introduced in Sect. 2, a DDG vertex is fixed if it is either a predecessor or a successor of every other vertex in the DDG, that is, if there is no other vertex that is independent of it. When a fixed vertex is found, Montemanni et al. decompose the DDG into two sub-problems: one consisting of the fixed vertex's predecessors and one consisting of its successors. The algorithm may also introduce fixed vertices by adding artificial edges between independent DDG nodes and splitting the problem into two sub-problems, one sub-problem for each direction of the added edge.

A simple exact algorithm, using a branch-and-bound approach for the SOP was proposed by Karan and Skorin-Kapov [13]. The lower bound used in that algorithm as

Author's personal copy

a pruning technique is the same as the incoming edge-based lower bound described in Sect. 5.2 of the present article. The algorithm is shown to perform well on relatively small instances (the largest instance size used in the experiments is 27).

In addition to the exact algorithms for the SOP, precise heuristics have been proposed [1,15]. The best known heuristic for the SOP problem is believed to be the Hybrid Ant System SOP (HAS-SOP) algorithm proposed by Gambardella and Dorigo [15]. This algorithm is based on the Ant Colony Algorithm that has been shown to perform well on the pure TSP [8].

Compared to existing exact solutions to the SOP in operational research, the branch-and-bound algorithm proposed in the present article is simpler than the branch-and-cut algorithm proposed by Ascheuer et al. [2] and the branch-and-bound algorithm proposed by Montemanni et al. [16], because it does not require an MILP solver. It also comprises a much more comprehensive set of pruning techniques than those used in the algorithm of Karan and Skorin-Kapov [13]. The history utilization technique described in the present article (see Sect. 5.5) is a generalization of the history domination technique proposed by Shobaki et al. [20–23].

4 Heuristic technique

In this section we describe the Nearest Neighbor heuristic that was used in previous work on switching energy minimization [18,25]. The NN heuristic is probably the simplest and most intuitive heuristic for solving the TSP and the PCTSP. In spite of its simplicity, an experimental study by Charikar et al. [4] shows that this heuristic performs relatively well in practice. Charikar et al.'s study, however, shows that the PCTSP is very hard to approximate, which means that it is unlikely to find a heuristic that works well on all instances.

```
Nearest_Neighbor_SOP (CG, DDG, N )
1   Sort the edges in the adjacency list of each CG node by edge weight in ascending order
2   for each node u in the CG
3       u.visited = false
4       u.depCnt = number of u's immediate predecessors in the DDG
5   x = x0 // The current node is initially a randomly selected node from the DDG's root nodes (a node with depCnt=0)
6   i = 1 // The number of visited nodes is initially 1
7   x.visited = true // Add x to the path
8   While i < N // while there are unvisited nodes
9       for each dependent y in the adjacency list of x in the DDG
10          y.depCnt = y.depCnt -1 // now one dependency of each of x's dependents has been satisfied
11      for each neighbor y in the sorted adjacency list of x in the CG
12          if y.visited == false AND y.depCnt==0 // pick the nearest neighbor that is ready and has not been visited
13              x = y // y's nearest neighbor has now become the current node
14              i = i + 1
15              x.visited = true // Add x to the path
```

Alg 1. Nearest-neighbor heuristic for the SOP

The NN heuristic may be described as follows. First, a root instruction in the DDG is selected as the start node. Then a Hamiltonian path is constructed incrementally by considering the current node's neighbors and augmenting the path with the neighbor at the other end of the minimum-weight outgoing edge (the nearest neighbor) provided that that neighbor is both unvisited and *ready*. A neighbor is ready if all of its DDG predecessors (if any) have been placed in the current partial path (schedule).

Author's personal copy

The pseudo-code for the nearest neighbor heuristic is shown in Alg 1. The code assumes that the cost graph (CG) and the DDG are represented by adjacency lists. Each graph has N nodes. The first step in the algorithm is sorting each node's adjacency list in the CG by edge weight in ascending order (Line 1). The *for* loop on Lines 2 to 4 initializes each node as unvisited and sets its *depCnt* to the number of its immediate predecessors in the DDG, that is, the number of nodes on which it depends. A node with a *depCnt* of zero is a root node in the DDG. In the code of Lines 5 to 7, one of these root nodes is selected at random as the start node. Each iteration of the main *while* loop starting on Line 8 adds one node to the path until all nodes have been visited (i = N). The *for* loop of Lines 9 to 10 updates the *depCnt* of the current node's successors (dependents) in the DDG. By visiting the current node, all of its dependents will have one of their dependencies satisfied. The *for* loop of Lines 11 to 15 goes through the current node's adjacency list, which is sorted by edge weight, and picks the first neighbor that is both unvisited and ready (its *depCnt* = 0) to be the new current node. The main loop starting on Line 8 then repeats until all nodes have been visited (i=N).

To illustrate the NN heuristic, we apply it to the example in Fig. 1. Initially, Instructions A and C are ready, because they are root instructions. Assuming that we first choose to schedule Instruction C, the ready instructions after scheduling Instruction C will be A and D. Instruction B won't be ready, because it is dependent on Instruction A, and Instruction A has not been scheduled yet. The NN heuristic will then select Instruction A to schedule next, because the weight of Edge (C, A) is smaller than the weight of the Edge (C, D) (A is C's nearest neighbor). After scheduling A, Instructions B and D will be ready. The NN heuristic will then select Instruction B, because it is A's nearest neighbor. Finally, Instruction D will be scheduled. This will produce the schedule (C, A, B, D) with cost 16. This is not optimal, because the schedule (C, D, A, B) is the optimal schedule with cost 12.

5 Branch-and-bound algorithm

In this section we describe the proposed branch-and-bound algorithm for solving the switching energy minimization problem, with a detailed description of each pruning technique.

5.1 Overview

The algorithm first constructs an initial feasible schedule using some heuristic technique, and then it computes a lower bound on the optimal schedule's cost using the lower bound techniques described below. The lower bounds computed in this stage before branch-and bound enumeration are called *static lower bounds* (SLBs) to distinguish them from the *dynamic lower bounds* (DLBs) that are computed during enumeration. Two types of lower bound techniques are used: edge-based lower bounds and minimum-spanning-tree-based (MST-based) lower bounds. These lower bound techniques are described in subsequent sub-sections. The cost of the heuristic schedule, which is an upper bound on the optimal cost, is then computed and compared to the static lower bound. If the heuristic cost is equal to the static lower bound, the heuristic schedule is optimal and the algorithm terminates. If the heuristic cost is greater than

Author's personal copy

the static lower bound, the branch-and-bound enumerator is invoked to perform an exhaustive search for an optimal schedule. At that point, the heuristic schedule is the *current best schedule* and its cost is the *current best cost*.

The enumerator performs an exhaustive search for an optimal schedule using a branch-and-bound approach. A branch-and-bound enumerator traverses a tree, called the *enumeration tree*, in which internal nodes represent partial solutions and leaves represent complete feasible solutions. In the proposed algorithm, traversal is done in a depth-first manner. In this traversal, only the nodes on the *active path* are physically present in memory. The active path is the path from the root to the current node. Accordingly, the maximum number of tree nodes that need to be stored in memory is a linear function of the input size. The enumerator constructs a feasible schedule incrementally by adding one instruction at a time. Initially, at the tree's root node, the schedule is empty. Adding one instruction leads to branching from the current node to a new node at the next level in the enumeration tree. The path from the root node to any given tree node represents a *partial schedule,* called the *prefix schedule* at that node. Each leaf node in the enumeration tree will then represent a complete feasible schedule. A complete feasible schedule *below* any non-leaf node x is a concatenation of x's prefix schedule and a *suffix schedule* below x, which is a permutation of the instructions that have not yet been scheduled at x. When any non-leaf tree node is reached, the prefix schedule is fixed and the enumerator searches the sub-tree below that node to find the best suffix schedule in that sub-tree.

At any tree node, there are generally multiple options to add to the current partial schedule. Each ready instruction is an option. Since the algorithm performs an exhaustive search, all ready instructions must be explored. In this work, ready instructions are explored in ascending edge-weight order. That is, if Instruction x is the last instruction in the current partial schedule, x's nearest neighbor is considered first, followed by the second nearest neighbor, and so on.

```
Enumerate(i) // i is the depth in the enumeration tree
1   if (optimalFound) return // if global optimality flag set, no need to continue
2   if (i == n+1) // if at a leaf node with a complete schedule
3       ProcessCompleteSolution() // check if it is better than the best solution
4       return
5   explore = CheckNewNode() // apply pruning techniques
6   if (explore == false) return //backtrack
7   for each option at current node
8       StepForward() // pick an instruction to schedule next and step to the next level in the tree
9       Enumerate(i+1) // repeat the same process at the next level in the tree

ProcessCompleteSolution()
    if (cost (currentSolution) < cost(bestSolution)) bestSolution = currentSolution
   if (cost(bestSolution) == staticLowerBound) optimalFound = true

CheckNewNode()
    // Apply pruning techniques to check if the current node may have a lower cost solution below it
    if (CheckHistory() == false) return false // history utilization
    if (ComputeOEDLB() == false) return false //outgoing edge dynamic lower bounds
    if (ComputeIEDLB() == false) return false //incoming edge dynamic lower bounds
    if (ComputeMSTDLB() == false) return false // MST dynamic lower bounds
    return true // if no pruning technique leads to pruning, then the current node may have a better solution
```

Alg 2. Branch-and-bound algorithm for the SOP

Author's personal copy

Fast enumeration requires efficient *pruning techniques*, which minimize the number of explicitly enumerated tree nodes. A pruning technique is a test that is performed at each tree node to determine if it is possible to find a feasible schedule below that node with a lower cost than the current best cost. If any pruning technique determines that it is not possible to find a lower cost feasible schedule below the current tree node, the enumerator undoes the last decision (un-schedules the last instruction scheduled) and backtracks to the previous tree node, where it attempts another candidate instruction, and so on. This backtracking effectively prunes the sub-tree below the current node.

In the proposed algorithm, the pruning techniques applied at each tree node are: edge-based dynamic lower bounds (EDLBs), MST-based dynamic lower bounds (MST DLBs) and history utilization. Note that edge-based and MST-based lower bounds are used both as static lower bounds to filter out easy instances before enumeration and as dynamic lower bounds to prune redundant tree nodes during enumeration. In the next sub-sections we describe these lower bound and pruning techniques in detail. Pseudo-code for the high-level branch-and-bound algorithm is listed in Alg 2. The pruning techniques applied in the CheckNewNode routine are explained in the subsequent sub-sections.

The size of the solution space is, in the worst case, a factorial function of the input size (all possible permutations of the instructions). The pruning techniques used in the algorithm significantly reduce the size of the solution space that is explicitly enumerated, but there is no provable polynomial upper bound on the number of enumerated nodes; otherwise, we would be resolving the P=NP question [7]. Therefore, the enumerator may not complete searching the solution space within reasonable time for some instances. In practice, the enumerator is given a certain time limit. If it does not find a provably optimal solution within this time limit, it reports the current best schedule, which may be very close to the optimal solution in many cases.

5.2 Edge-based lower bounds

A feasible schedule corresponds to a Hamiltonian path in the cost graph. A Hamiltonian path with n nodes has n−1 edges. Therefore, a lower bound on the optimal cost may be computed by summing the lowest n−1 edge weights in the cost graph. A more precise lower bound may be computed by identifying the *minimum outgoing edge weight* (MOEW) for each node in the cost graph and summing the lowest n−1 minimum outgoing edge weights. Given a node x in the cost graph, an edge (x, y) is an *outgoing edge* for Node x if Node y may appear immediately after Node x in some feasible schedule. Node y may appear immediately after x in a feasible schedule if x and y are independent nodes in the DDG or if y is an immediate successor of x in the DDG. If y is a non-immediate successor of x in the DDG (there is at least one other node on the path from x to y) or if y is a predecessor of x, y can never appear immediately after x in any feasible schedule. The MOEW of a node is the minimum weight of any outgoing edge for that node. It is noted that in any Hamiltonian path, all nodes must have outgoing edges except leaf nodes in the DDG. Therefore, an *outgoing edge-based lower bound* (or simply an outgoing edge lower bound) may be computed by summing all n MOEWs and then subtracting the largest MOEW among all leaf node MOEWs.

Author's personal copy

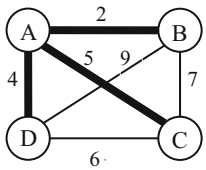
Fig. 2 Static lower bounds (SLBs) for the instance shown in Fig. 1

(a) Edge-based lower bound

Node	A	B	C	D
MOEW	2	7	5	4
MIEW	4	2	5	4

Outgoing edge SLB = 2 + 7 + 5 + 4 – 7 = 11
Incoming edge SLB = 4 + 2 + 5 + 4 – 5 = 10
Bidirectional edge SLB = Max (11, 10) = 11

(b) MST-based lower bound



MST SLB = 11.

Since the switching energy problem is asymmetrical, a different edge-based lower bound may be computed by considering *incoming edge* weights. An edge (y, x) in the cost graph is an incoming edge for Node x if y may appear immediately before x in a feasible schedule, that is, if x and y are independent nodes in the DDG or if y is an immediate predecessor of x in the DDG. The minimum incoming edge weight (MIEW) of a node is the minimum weight of any incoming edge for that node. All nodes must have incoming edges except root nodes in the DDG. Therefore, an *incoming edge-based lower bound* (or simply an incoming edge lower bound) may be computed by summing all n MIEWs and then subtracting the largest MIEW among root node MIEWs. Taking the larger of the incoming edge-based lower bound and the outgoing edge-based lower bound gives the *bidirectional* edge-based lower bound.

Figure 2a shows the outgoing and incoming edge-based static lower bounds for the instance of Fig. 1. For example, the outgoing edges for Node B are edges (B, C) and (B, D) with weights 7 and 9, respectively. Edge (B, A) is not an outgoing edge for Node B, because, A is a predecessor of B in the DDG, and therefore A can never appear immediately after B in any feasible schedule. The MOEW of Node B is then 7, which is the weight of Edge (B, C). The leaf nodes in the DDG are Nodes B and D with MOEWs of 7 and 4, respectively. Since these are the only nodes that may not have outgoing edges in a feasible schedule, the outgoing edge SLB for this instance is the sum of all MOEWs minus 7. Similarly, MIEWs are computed, and following a similar procedure leads to an incoming edge SLB of 10. The bidirectional edge SLB for this instance is the larger of the outgoing edge SLB and the incoming edge SLB, which is 11.

5.3 MST-based lower bounds

The minimum spanning tree of a weighted undirected graph with n nodes is a minimum-cost tree that connects all the nodes in the graph [7]. Similar to a Hamil-

Author's personal copy

tonian path, an MST consists of n-1 edges that connect all the nodes in an n-node graph. Unlike a Hamiltonian path, an MST does not have to be a simple path that visits each node exactly once (for example, an MST could be star shaped). Therefore, the MST problem is less constrained than the Hamiltonian path problem and the cost of the MST is a lower bound on the cost of the Hamiltonian path [11]. There are efficient algorithms for solving the MST problem, including Prim's algorithm [7]. Using a heap priority queue, Prim's algorithm solves the problem in O(mlogn) time, where n is the number of nodes and m is the number of edges in the graph.

If the cost graph is a directed weighted graph, an MST-based lower bound can still be computed by transforming the directed graph into an undirected graph as follows. For each pair of directed edges (x, y) and (y, x) with weights w_1 and w_2, respectively, consider the following four cases.

1. If y is an immediate successor of x in the DDG, remove Edge (y, x) and replace the directed edge (x, y) with an undirected edge (x, y) with the same weight w_1.
2. If x is an immediate successor of y in the DDG, remove Edge (x, y) and replace the directed edge (y, x) with an undirected edge (y, x) with the same weight w_2.
3. If x and y are dependent nodes in the DDG but none of them is an immediate successor of the other, remove both (x, y) and (y, x).
4. If x and y are independent, replace the two directed edges with one undirected edge (x, y) with weight w that is equal to the minimum of w_1 and w_2.

A standard MST algorithm can then be applied to the resulting undirected graph. Clearly, this will give a valid lower bound on the minimum Hamiltonian path cost. However, this lower bound is not expected to be as tight as that computed for an undirected cost graph. Intuitively, the more asymmetric the weighted graph is, the looser this lower bound will be.

Figure 2b shows the MST for the instance of Fig. 1. This MST gives a lower bound of 11, which happens to be equal to the bidirectional edge SLB in this case. Clearly, the MST-based lower bounds and the edge-based lower bounds will not always be equal. Each lower bound technique captures different constraints and may therefore give a different value. More specifically, edge-based lower bounds capture precedence constraints, while the MST is defined for an undirected graph. Furthermore, the MST may have multiple incoming or outgoing edges for the same node, while the outgoing (incoming) edge-based lower bound ensures that each node has only one outgoing (incoming) edge. On the other hand, an edge-based lower bound may use the same edge multiple times, while in an MST an edge can only be used once. Therefore, the two lower bound techniques complement each other.

5.4 Dynamic lower bounds as pruning techniques

The edge-based and MST-based lower bounds described in the previous sections can be used as pruning techniques during enumeration. When the enumerator makes a scheduling decision and steps forward to a new tree node, both lower bounds are updated according to the decision that has been made. This update often results in a larger (tighter) lower bound at the new node (a DLB). The DLB at the new node is compared with the current best cost. If the DLB at the new node is greater than or

Author's personal copy

equal to the current best cost, no better schedule can be found below the new node, and the enumerator backtracks. This prunes the whole sub-tree below the new node.

At any new node in the enumeration tree, DLBs are computed by adding the cost of the prefix schedule (the *prefix cost* or the *fixed cost*) to a lower bound on the cost of the suffix schedule (the *suffix cost*). The prefix cost is simply the sum of the weights of the edges that have been scheduled in the current node's prefix schedule. A lower bound on the suffix cost is computed by considering the sub-graph consisting of the nodes and edges that have not been scheduled. In computing the edge-based DLBs, a lower bound on the suffix cost is computed by summing the MOEWs or MIEWs for the instructions whose outgoing or incoming edges have not been fixed in the prefix schedule. In computing these updated MOEWs or MIEWs, the edges that have been selected in the prefix schedule are excluded, thus resulting in a tighter DLB. In computing the MST DLBs, a lower bound on the suffix cost is computed by finding the MST for the sub-graph consisting of unscheduled instructions and adding to that the minimum cost for linking the prefix schedule to unscheduled instructions. The minimum linking cost is equal to the MOEW of the last scheduled instruction in the current node's prefix schedule.

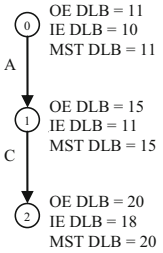
Figure 3 shows how DLBs are computed and used as pruning techniques in scheduling the instance of Fig. 1. Figure 3a shows part of the enumeration tree with three nodes. At Node 0 (the root node), no instruction has been scheduled and all DLBs are equal to the corresponding SLBs. At Node 1, the decision of scheduling Instruction A has been made, and the DLBs are updated as detailed in Fig. 3b. At Node 2, Instruction C is scheduled, which produces the prefix schedule (A, C). Comparing Fig. 3b with Fig. 2a shows that scheduling Instruction A leads to updating the MOEWs of Instructions C and D. In Fig. 2a it is assumed that the minimum-weight outgoing edges for Instructions C and D are Edges (C, A) and (D, A), respectively. With the scheduling of Instruction A, these edges are no longer valid outgoing edges for C and D. Therefore, the next minimum-weight outgoing edge is taken for each Instruction (Edge (C, D) for C and Edge (D, B) for D). This leads to a tighter outgoing edge DLB of 15. Similarly, the MIEWs are updated as shown in the third row of the table in Fig. 3b. It is noted here that by selecting Instruction A to be scheduled first, Instruction A will be the node with no incoming edge. Therefore, the incoming edge DLB is the sum of the MIEWs of the remaining instructions.

The last part of Fig. 3b shows how the MST DLB is computed. The MST DLB is the sum of three components: the prefix cost, the MST cost of Sub-graph BCD (unscheduled instructions), and the MOEW of A (the last scheduled instruction). This sum gives an MST DLB of 15.

When Instruction C is scheduled next at Node 2, further tightening of the DLBs can be done as shown in Fig. 3a and detailed in Fig. 3c. The fact that Edge (A, C) is now part of the prefix schedule leads to changing the MOEW of Instruction A to the weight of Edge (A, C), which is 5. Furthermore, Edge (B, C) is no longer a valid outgoing edge for Instruction B. Therefore, the MOEW for Instruction B is updated to the weight of Edge (B, D), which is 9. These two updates lead to an outgoing edge DLB of 20. Similarly, the MIEWs are updated as shown in the table of Fig. 3c. The figure also shows how the MST DLB is updated after the scheduling of Instruction C. The MST DLB is now equal to the sum of the prefix cost (the weight of Edge (A, C)),

Author's personal copy

(a) Enumeration tree and DLBs after scheduling Instruction A then Instruction C

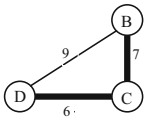


(b) DLBs after scheduling Instruction A

Edge-based DLBs

Instruction	A	B	C	D
MOEW	2	7	6	9
MIEW	None	2	5	4

Outgoing edge DLB = 2 + 7 + 6 + 9 – 9 = 15
Incoming edge DLB = 2 + 5 + 4 = 11
Bidirectional edge DLB = Max (15, 11) = 15



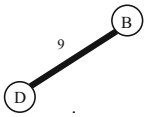
MST DLB:
Prefix cost = 0 (no edges selected at this point)
MST cost of sub graph BCD (unscheduled instructions) = 6 + 7 = 13
MST DLB = prefix cost + MST cost (BCD) + MOEW(A) = 0 + 13 + 2 = 15

(c) DLBs after scheduling Instruction C

Edge-based DLBs

Instruction	A	B	C	D
MOEW	5	9	6	9
MIEW	None	7	5	6

Outgoing edge DLB = 5 + 9 + 6 + 9 – 9 = 20
Incoming edge DLB = 7 + 5 + 6 = 18
Bidirectional edge DLB = Max (20, 18) = 20



MST DLB:
Prefix cost = cost of Edge (A, C) = 5
MST cost of sub-graph BD (unscheduled instructions) = 9
MST DLB = prefix cost + MST cost(BD) + MOEW(C)= 5 + 9 + 6 = 20

Fig. 3 Dynamic lower bounds

the MST cost of Sub-graph BD and the MOEW of Instruction C (the last scheduled instruction). As shown in the Figure, this sum gives an MST DLB of 20.

Updating edge-based lower bounds at each new tree node can be implemented efficiently by maintaining two weight-sorted adjacency lists for each node: one adjacency list for outgoing edges and another adjacency list for incoming edges. These adjacency lists must account for precedence constraints. More specifically, Instruction y appears in the outgoing adjacency list of Instruction x if and only if y is an immediate successor of x or x and y are independent instructions in the DDG. A similar condition applies to the incoming adjacency lists. It is important to note that the adjacency lists only need to be sorted once before enumeration starts (a preprocessing step). During enumeration, a pointer to the current outgoing (or incoming) edge is kept in each adjacency list. When the enumerator steps forward, some of these pointers may move down their adjacency lists, and when the enumerator backtracks, all pointers restore their previous positions. This allows dynamic edge-based

Author's personal copy

lower bounds to be computed fairly quickly. The experimental results will confirm this.

5.5 History utilization

Shobaki and Wilken [20] introduced a technique called *history-based domination* that they used to do efficient branch-and-bound instruction scheduling of superblocks. A more detailed description of this technique appears in Shobaki's dissertation [21]. In this section, we present a generalized form of this technique that we call *history utilization*. Unlike the original history-based domination technique, which is solely used to prune tree nodes that do not have below them better solutions than the current best solution, the generalized history utilization technique presented in the current article may take advantage of history information (solution to a previously solved related sub-problem) to construct a better solution than the current best solution without having to explore the sub-tree below the current node.

The main idea in history utilization is to store information about previously processed tree nodes (solved sub-problems) in a *history table* and take advantage of that information to speed up the processing of *similar* tree nodes that may be encountered later, which is essentially a dynamic programming approach [7]. Two tree nodes x and y are similar if the set of scheduled instructions in x's prefix schedule is equal to the set of scheduled instructions in y's prefix schedule and the last instruction in both partial schedules is the same. If two nodes x and y are similar, the remaining sub-problem to be solved below x is the same as the remaining sub-problem to be solved below y. For the switching cost minimization problem addressed in this article, if the current tree node is similar to a previously visited tree node (*history node*), the constraints on the remaining sub-problems below both nodes will be exactly the same. Thus, any schedule that is feasible below the current node was also feasible below the history node. This may be used to speed up the processing of the current node as follows.

If the prefix cost at the current node is greater than or equal to the prefix cost of a similar history node, the current node cannot have below it a better schedule than the best schedule below the history node, and the sub-tree below the current node may be pruned away. That's because the total cost of the best complete schedule below the current node is equal to the cost of the scheduled instructions (prefix cost) plus the cost of the optimal solution to the remaining sub-problem (optimal suffix cost), and the optimal suffix cost must be the same below both nodes, because the constraints on the remaining sub-problems are identical.

If, on the other hand, the prefix cost at the current node is smaller than the prefix cost at the history node, the current node will have below it a better schedule than the best schedule below the history node. That schedule, however, will not necessarily be better than the current best schedule. The difference between the prefix cost at the history node and the prefix cost at the current node represents the *possible improvement* that can be achieved at the current node relative to the history node. The *required improvement* for constructing a better schedule than the current best schedule is equal to the difference between the best total cost below the history node and the current

Author's personal copy

best cost. If the possible improvement at the current node is less than or equal to the required improvement, the best schedule below the current node will not be better than the current best schedule, and the sub-tree below the current node may be pruned. Note that the best total cost below the history node may or may not be known exactly. If the history node has a complete suffix schedule below it, the best total cost below the history node will be known exactly. If, on the other hand, DLB-based pruning has occurred before reaching any leaf node below the history node, no complete suffix schedule will have been constructed below the history node, and only a lower bound on the best total cost below the history node will be known. Computing the difference between this lower bound and the current best cost gives a lower bound on the required improvement. If the possible improvement at the current node is less than or equal to a lower bound on the required improvement, the sub-tree below the current node may be pruned.

```
// This routine returns true if the sub-tree below the current node must be
// explored and false if it may be pruned.
// If the history node has a suffix schedule below it, the DLB of the history node
// is set to the total cost of the best schedule below it

HistoryUtilization (currentNode, histNode) {
1   if currentNode.prefixCost ≥ histNode.prefixCost
2       return FALSE
3   imp = histNode.prefixCost – currentNode.prefixCost
4   if imp ≤ histNode.DLB – bestCost
5       return FALSE
6   histNode.prefixSched = currentNode.prefixSched
7   histNode.prefixCost = currentNode.prefixCost
8   if histNode.suffixSched != NULL
9       bestSched = currentNode.prefixSched +histNode.suffixSched
10      bestCost = currentNode.prefixCost +histNode.suffixCost
11      histNode.DLB = bestCost
12      return FALSE
13 return TRUE
 }
```

Alg 3. History utilization.

Finally, if the prefix cost at the current node is smaller than the prefix cost at the history node and the possible improvement at the current node is greater than the required improvement (or its lower bound), the current node may have below it a better schedule than the current best schedule. If the history node has a complete suffix schedule, a new best schedule may be constructed directly by concatenating the suffix schedule below the history node to the current node's prefix schedule and no further enumeration is needed below the current node. If the history node does not have a suffix schedule, the current node must be explored to find the best suffix schedule below it. The pseudo-code for the history utilization algorithm is listed in Alg 3.

In order for the history utilization algorithm to work efficiently, the implementation of the history table must support quick search for a previously visited similar node. In our implementation, a hash table is used to store previously visited nodes. The combination of scheduled instructions as well as the last scheduled instruction at a given tree node are encoded as a bit vector, which is used as the *key* in the hash table.

Author's personal copy

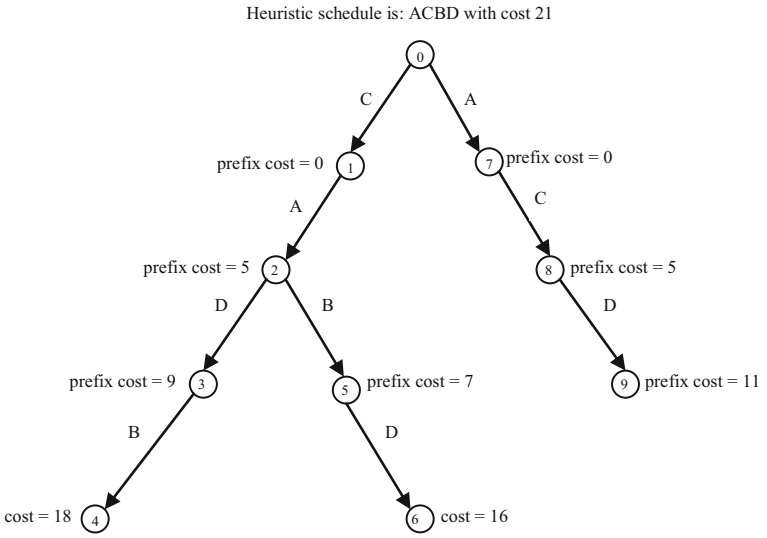


Fig. 4 History utilization example

Figure 4 shows an example illustrating history utilization. In this example, it is assumed that the initial heuristic schedule constructed before enumeration is (A, C, B, D) with cost 21. First, we assume that the nodes are visited in the order shown in Fig. 4 (depth-first order from left to right). In this order, Node 3 is visited before Node 6 and Node 6 is visited before Node 9. To fully illustrate the algorithm, we will also consider different orders for visiting the three nodes. The order of visiting these three nodes is particularly interesting, because Nodes 3 and 9 are similar nodes and Node 6 is a leaf node at which a complete schedule (C, A, B, D) with cost 16 is discovered. Depending on the branch selection strategy, these nodes may be visited in different orders. For illustration, we consider the following three orders.

Case 1: Node 3 is visited first, then Node 6 then Node 9. In this case, Node 4, which is a leaf node with complete schedule (C, A, D, B), will be visited immediately after Node 3. At Node 4, the enumerator will update the current best schedule to Schedule (C, A, D, B) and the current best cost to 18. It will then backtrack to Node 3. At that point, the sub-tree below Node 3 will have been fully explored, and Node 3 will be stored in the history table. The enumerator will then continue to traverse the tree in depth-first order until it reaches Node 6. At Node 6, the current best schedule becomes Schedule (C, A, B, D) with cost 16. When Node 9 is reached later on, the history table will be searched for a previously visited similar node and Node 3 will be found. The history utilization algorithm will then be applied to the two similar nodes (Node 3 as a history node and Node 9 as the current node). In this case, the condition on Line 1 of the algorithm will be satisfied, because the prefix cost of Node 9 is larger than the prefix cost of Node 3, and the sub-tree below Node 9 will be pruned.

Case 2: Node 6 is visited first, then Node 9 then Node 3. In this case, when Node 6 is visited, the complete schedule (C, A, B, D) with cost 16 will be discovered first and stored as the current best schedule. When Node 9 is visited later on, a DLB will be computed by adding the prefix cost of 11 to the MOEW of D, thus giving a total

Author's personal copy

DLB of 20. Since this DLB is larger than the current best cost of 16, the sub-tree below Node 9 will be pruned away, and Node 9 will be stored in the history table with a prefix cost of 11 and a suffix cost DLB of 9. When Node 3 is visited, similarity with node 9 in the history table will be detected, and the history utilization algorithm will be applied to Node 3 (as the current node) and Node 9 (as a history node). The prefix cost of 9 at Node 3 is smaller than the prefix cost of 11 at Node 9. However, the possible improvement is only 2, which is smaller than the required improvement of 4 (the difference between the total history DLB of 20 and the current best cost of 16). This implies that although Node 3 has a better schedule than the best schedule below Node 9, that schedule will at best have a cost of 18, which is still larger than the current best cost. Therefore, the condition on Line 4 of Alg. 3 will be satisfied and the sub-tree below Node 3 will be pruned.

Case 3: Node 9 is visited first, then Node 3 then Node 6. In this case, the total DLB of 20 at Node 9 will be less than the current best cost at that point, which will be the heuristic cost of 21. Therefore, no DLB-based pruning will occur at Node 9, and the sub-tree below Node 9 will be explored. Node 10 (a leaf node not shown in the figure) will be visited immediately afterwards, and the complete schedule (A, C, D, B) with cost 20 will be constructed at that node and stored as the current best schedule. When the search returns to Node 9, Node 9 will be stored in the history table with a prefix cost of 11 and a suffix cost of 9 (note that Node 9 will have an actual suffix cost in this case, not a suffix-cost lower bound). The suffix schedule consisting of Instruction B will also be stored in Node 9's history entry as the best suffix schedule below Node 9. The enumerator will then reach Node 3 and the history utilization algorithm will be applied to Node 3 as the current node and Node 9 as a similar history node. Since the current prefix cost is smaller than the history prefix cost in this case, Lines 6 through 12 in Alg2 will be executed, and the best suffix schedule below Node 9 will be concatenated to the prefix schedule at Node 3 to form the current best schedule (C, A, D, B) with cost 18. Hadn't the history entry of Node 9 had a best suffix schedule stored in it, the condition on Line 8 would not have been satisfied and the sub-tree below Node 3 would have been explored to find the best suffix schedule. This would have been the case if a schedule with cost less than 20 but greater than 18 had been discovered before visiting Node 9 (not possible in this example), thus leading to DLB-based pruning at Node 9.

6 Experimental results

The algorithm described in this paper was implemented and integrated into the LLVM Compiler (Version 3.3) as an alternate post-allocation scheduler. The target architecture in all experiments was ARM. Cross-compilation for ARM was performed on an Intel i7-3632QM processor with 4 GB of memory. The LLVM backend (llc) was invoked with the command line options: -O3 -march=arm -mcpu=cortex-A5. The operating system was Ubuntu 12.04. The test cases were 14 benchmarks from MiBench.[1]

[1] Some MiBench benchmarks were excluded due to building problems.

Author's personal copy

6.1 Input data

In this section, we present some statistical information about the benchmarks that were used to in the experimental evaluation. Since scheduling in LLVM is done locally at the basic block level, each input instance to the scheduling algorithm is a basic block. Table 1a shows the number of basic blocks and their size distribution in each benchmark. Table 1b shows basic block size distribution across all benchmarks.

Table 1 Basic block (BB) distribution in MiBench

(a) Basic block counts and sizes in each benchmark

Benchmark	BB count	Min size	Max size	Avg. size
Adpcm	56	3	29	6
Bitcount	77	3	37	6
CRC32	26	3	13	5
Dijkstra	106	3	16	6
FFT	153	3	23	5
GSM	1,198	3	352	7
Jpeg	5,924	3	137	6
Patricia	145	3	13	4
Qsort	51	3	17	5
Rijndael	164	3	598	18
Sha	59	3	20	7
Stringsearch	220	3	16	5
Susan	538	3	199	9
Typeset	22,007	3	435	7
ALL	30,724	3	598	7

(b) Basic block size distribution across all benchmarks

BB size range	BB count	Percentage
[1–15]	28,376	92.358
[16–50]	2,204	7.174
[51–100]	98	0.319
[101–150]	15	0.049
[151–200]	5	0.016
[201–250]	20	0.065
[251–300]	2	0.007
[301–350]	0	0
[351–400]	1	0.003
[401–450]	1	0.003
[451–500]	0	0
[501–550]	0	0
[551–600]	2	0.007

Author's personal copy

There are a total of 30,724 basic blocks in the data set. Although the numbers in Table 1a show that most basic blocks are relatively small (the average basic block size is 7 instructions), Table 1b shows that there is a significant number of large basic blocks. There are thousands of basic blocks in the [16–50] size range, which, as shown by the results in the subsequent sections, are not easy to schedule optimally using heuristic approaches. There are 98 basic blocks in the size range [51–100] and 46 basic blocks with sizes over 100. The heuristic results in subsequent sections show that the NN heuristic fails to produce optimal solutions for a significant percentage of these blocks and that the proposed branch-and-bound algorithm produces significant improvements in switching cost relative to the heuristic.

6.2 Heuristic results

In this section, we evaluate the performance of the NN heuristic relative to the critical-path (CP) heuristic, which is a commonly used heuristic for minimizing the schedule length [6]. Since the CP heuristic does not account for the switching cost, it is used here as an energy-agnostic baseline.

Table 2 shows the total switching cost produced by each heuristic for each benchmark. The table also shows the percentage reduction in switching cost for the NN heuristic relative to the CP heuristic. The switching costs shown in the table were computed based on the Hamming distance model described in Sect. 2. The Hamming distances were computed using the *real* instruction encodings generated by the LLVM Compiler for the ARM target. The number shown for each benchmark is the total switching cost for the entire benchmark.

Table 2 Heuristic performance

Benchmark	CP	NN	
	SW cost	SW cost	Percentage reduction
Adpcm	1,401	1,207	13.85
Bitcount	1,574	1,256	20.2
CRC32	433	351	18.94
Dijkstra	1,818	1,494	17.82
FFT	2,040	1,764	13.53
GSM	32,488	30,146	7.21
Jpeg	116,960	105,827	9.52
Patricia	1,794	1,621	9.64
Qsort	772	652	15.54
Rijndael	22,731	20,174	11.25
Sha	1,414	1,307	7.57
Stringsearch	3,407	3,129	8.16
Susan	27,162	23,768	12.5
Typeset	567,702	495,272	12.76
TOTAL	781,696	687,968	11.99

Author's personal copy

The results in Table 2 show that using a switching-cost-aware heuristic significantly reduces the switching cost compared to using a switching-cost-agnostic heuristic. The NN heuristic gives an overall reduction of 12 % in switching cost relative to the CP heuristic. The maximum percentage reduction is seen on the Bitcount benchmark, where NN gives a reduction of approximately 20 % in switching cost.

6.3 Branch-and-bound pruning techniques

In this section we evaluate the performance of the branch-and-bound algorithm described in this paper using different combinations of pruning techniques. First, the *easy* basic blocks were filtered out by comparing the NN heuristic cost to the static lower bound. For 17,546 basic blocks (57.1 %), the heuristic cost was equal to the static lower bound, and those blocks were filtered out. The static lower bound was computed by taking the larger of the bidirectional edge-based lower bound and the MST-based lower bound. The remaining 13,178 basic blocks (42.9 %) were passed to the branch-and-bound enumerator to search for an optimal schedule. The time limit was set to 10 ms per instruction. So, a basic block with 100 instructions was given a time limit of 1s.

First, we evaluate the different DLB techniques without history utilization (Table 3), and then we study the impact of history utilization (Table 4). Finally, we show some statistics about the branch-and-bound algorithm with and without history utilization using different DLB techniques (Table 5).

In Table 3, the relative performance of the different DLB techniques is evaluated by comparing the number of tree nodes (Table 3a) and the solution time (Table 3b) needed by each DLB technique, if used alone, to optimally schedule the same set of basic blocks. For a fair comparison, the set of basic blocks used in this experiment was the set of basic blocks that were scheduled optimally within the time limit using *any* DLB technique. This set consists of 12,866 basic blocks. History utilization was not applied in this experiment. The node counts in Table 3a and the solution times in Table 3b show that no DLB technique gives the best results in all cases. The last column shows the number of tree nodes (Table 3a) and the solution time (Table 3b) needed when all DLB techniques were applied and the best DLB at each node was used for pruning.

It is noted in Table 3a that the MST DLB is an effective pruning technique that optimally schedules the basic blocks with fewer tree nodes than those needed by the bidirectional edge DLB for 9 out of 14 benchmarks. It is also observed that for all benchmarks, the number of tree nodes needed when all DLB techniques are enabled is significantly less than the number of tree nodes needed when the bidirectional DLB is used alone. Both of these observations indicate that using the MST-based DLB gives better DLBs and thus better pruning than using edge-based DLBs. Examining the last column in Table 3b, however, shows that the significantly improved pruning achieved using all DLB techniques, including the MST DLBs, does not always lead to a faster solution time. In fact, the overall solution times shown in the last row of Table 3b are approximately the same using any DLB combination. This is attributed to the fact that although using more DLB techniques significantly reduces the number of tree nodes

Author's personal copy

Table 3 Enumeration to optimality using different DLB techniques without history utilization. Number of blocks = 12,866

Benchmark	Incoming edge DLB	Outgoing edge DLB	Bidirectional edge DLB	MST DLB	ALL DLBs
(a) Number of tree nodes needed to reach optimality using each DLB technique					
Adpcm	211,234	310,521	194,043	39,039	38,974
Bitcount	209,076	107,332	89,362	186,402	65,420
CRC32	2,995	2,369	2,203	1,318	995
Dijkstra	17,980	42,982	17,152	11,576	9,304
FFT	206,176	133,816	107,805	136,881	60,340
GSM	701,469	472,739	408,121	464,411	314,277
Jpeg	4,109,873	4,790,985	3,000,359	2,854,547	1,595,382
Patricia	1,444	1,648	1,226	828	743
Qsort	19,855	17,648	14,664	13,775	9,920
Rijndael	15,903	20,135	13,820	10,025	7,587
Sha	42,816	52,295	31,686	10,510	9,080
Stringsearch	3,375	3,383	2,868	2,485	2,179
Susan	294,421	309,478	222,205	269,800	157,274
Typeset	27,917,994	25,487,805	20,858,733	27,630,347	17,479,437
TOTAL	33,754,611	31,753,136	24,964,247	31,631,944	19,750,912
(b) Solution time in ms needed to reach optimality using each DLB technique					
Adpcm	90	150	100	40	50
Bitcount	90	90	60	120	70
CRC32	10	0	20	0	10
Dijkstra	10	60	20	10	30
FFT	80	80	70	80	30
GSM	470	320	310	330	390
Jpeg	2,410	3,110	2,280	2,070	1,710
Patricia	0	10	0	0	10
Qsort	20	10	10	10	0
Rijndael	10	20	20	20	0
Sha	0	20	20	40	10
Stringsearch	30	30	30	10	10
Susan	160	210	190	210	180
Typeset	14,810	15,810	15,910	16,360	17,300
TOTAL	18,190	19,920	19,040	19,300	19,800

Time limit = 10 ms/instr

needed to reach optimality, this pruning benefit is overweighed by the time needed to compute the additional lower bounds at each tree node. Careful examination of the numbers in Table 3a and b shows that the MST-based DLB is computationally more expensive than the edge-based DLBs.

Table 4a and b show the same node counts and solution times reported in Table 3a and 3b but with history utilization enabled. First, it is noted that the number of basic

Author's personal copy

Table 4 Enumeration to optimality using different DLB techniques with history utilization

Benchmark	Bidirectional	MST	ALL
(a) Number of tree nodes needed to reach optimality using each DLB technique			
Adpcm	143,544	7,117	7,057
Bitcount	15,283	15,637	14,348
CRC32	1,509	871	734
Dijkstra	9,004	4,724	4,440
FFT	9,701	9,786	7,513
GSM	512,223	436,215	423,768
Jpeg	2,974,404	2,789,523	2,514,313
Patricia	986	758	697
Qsort	2,758	1,956	1,867
Rijndael	53,934	52,974	51,750
Sha	11,256	5,044	4,654
Stringsearch	2,128	1,904	1,690
Susan	1,774,012	1,361,904	1,326,502
Typeset	4,751,192	3,936,130	3,674,818
TOTAL	10,261,934	8,624,543	8,034,151
(b) Solution time in ms needed to reach optimality using each DLB technique			
Adpcm	90	30	10
Bitcount	50	30	70
CRC32	0	0	0
Dijkstra	0	10	20
FFT	10	30	20
GSM	540	4,550	4,730
Jpeg	2,640	5,500	5,950
Patricia	10	10	0
Qsort	0	0	10
Rijndael	80	1,480	1,520
Sha	20	10	40
Stringsearch	30	20	20
Susan	1,370	4,320	4,450
Typeset	5,100	23,680	24,000
TOTAL	9,940	39,670	40,840

Number of blocks = 13,137. Time limit = 10 ms/instr

blocks that are solved optimally within the time limit by any DLB technique is now 13137, which means that 171 more basic blocks could be scheduled optimally within the same time limit with the enabling of history utilization. Comparing the number of enumeration tree nodes needed to reach optimality in Table 4a with the corresponding node counts in Table 3a shows a significant reduction in the number of tree nodes needed to reach optimality when history utilization is applied although more basic blocks are solved in Table 4a. With all DLB techniques enabled, the total number of tree nodes needed to optimally schedule all 12866 basic blocks in Table 3a is about 20

Author's personal copy

Table 5 Branch-and-bound statistics using different DLB techniques

	Stat	Bidirectional	MST DLB	ALL DLBs
(a) Without history utilization				
1	BBs enumerated	13,178	13,178	13,178
2	BBs timed out	281 (2.1 %)	279 (2.1 %)	267 (2.0 %)
3	BBs timed out and improved	249	249	238
4	BBs timed out and not improved	32	30	29
5	Largest BB solved optimally	158	158	158
6	Largest BB timed out and improved	598	435	435
7	Smallest BB timed out	14	17	14
(b) With history utilization				
1	BBs enumerated	13,178	13,178	13,178
2	BBs timed out	23 (0.2 %)	39 (0.3 %)	40 (0.3 %)
3	BBs timed out and improved	23	37	38
4	BBs timed out and not improved	0	2	2
5	Largest BB solved optimally	598	270	270
6	Largest BB timed out and improved	248	435	435
7	Smallest BB timed out	17	22	20

Time limit = 10 ms/instr

million nodes, while in Table 4a, only 8 million tree nodes were enough to optimally schedule 13137 basic blocks within the same time limit.

Comparing the solution times in Tables 3b and 4b should be done carefully, because 171 more basic blocks are included in the results of Table 4b. In spite of the increase in the number of basic blocks, the last row in Table 4b shows that when history utilization is combined with bidirectional edge-based DLBs, the overall solution time is much smaller than that obtained without history utilization (10 s compared to 19 s). These numbers lead to the conclusion that this particular combination of pruning techniques (history utilization combined with bidirectional edge-based DLBs) gives a significantly faster solution time than any other combination of pruning techniques.

Table 5 shows statistical information about the branch-and-bound algorithm using three different DLBs with and without history utilization. In spite of timing out before reaching optimality on some blocks, the enumerator may still produce better solutions than the heuristic solutions for some of these blocks. Row 3 in each table shows the number of blocks that timed out but were improved by the branch-and-bound algorithm relative to the NN heuristic, while Row 4 shows the number of blocks that timed out but were not improved relative to the NN heuristic.

Comparing Table 5a with Table 5b shows that history utilization gives a significant reduction in the number of timeouts. The results in this table suggest that the most effective combination of pruning techniques is history utilization with bidirectional edge-based DLBs, which is consistent with the conclusion drawn based on the results of Tables 3 and 4. With this combination of pruning techniques, only 23 basic blocks (0.2 %) timeout, and the largest basic block in the data set (the basic block with size

Author's personal copy

Table 6 Percentage contribution of each pruning technique

Pruning technique	% contribution without history	% contribution with history
Outgoing edge DLB	48.6	3.1
Incoming edge DLB	7.4	0.9
MST DLB	44.0	5.4
History utilization	0.0	90.6

598) was solved optimally within its time limit of approximately 6 s. The reduction of timeouts from 281 to 23 with history utilization means that using history utilization solved 92 % of the hard problems that could not be solved using the edge-based lower bounds alone. It is also noted that all 23 basic blocks that timeout were improved relative to the NN heuristic. So, with this combination of pruning techniques, 99.8 % of the enumerated blocks were solved optimally and the remaining 0.2 % were improved relative to the heuristic.

Row 7 shows that a basic block with only 17 instructions could not be solved optimally within its time limit of 170 ms using edge-based lower bounds and history utilization. The results in Sect. 6.5 will show that this small basic block can be solved optimally by slightly increasing the time limit.

Comparing the number of timeouts in Table 5a with those in Table 5b shows that applying the MST DLB reduces the number of timeouts when history utilization is disabled, while it increases the number of timeous when history domination is enabled. This is attributed to the fact that the MST DLB is computationally more expensive than the other pruning techniques (edge DLBs and history utilization), and when history utilization is enabled, it covers most of the pruning opportunities that are otherwise handled by the MST DLB technique.

To support this explanation, Table 6 shows the percentage contribution of each pruning technique with and without history utilization. Each entry in the table is the percentage of successful pruning operations across all benchmarks that were made by each pruning technique. The numbers in the last column show that, when history utilization is enabled, it becomes the dominant pruning technique that accounts for 90.6 % of the pruning, thus making the MST DLB less useful. It should be pointed out here that the time needed to apply history utilization at a given node is much less than the time needed to compute the MST-based DLB. Examining the code in Alg 3 shows that most cases are handled in O(1) time, assuming that the history table is a prefect hash table. The only case that takes O(n) time is the case of concatenating a history suffix schedule to the current prefix schedule to form a new best schedule.

In Table 6 it is noted that the outgoing edge DLB accounts for more pruning than the incoming edge DLB. This should not be used to draw a conclusion about the relative effectiveness of these two techniques, because the difference is simply attributed to the fact that the outgoing edge DLB is computed before the incoming edge DLB at each tree node (see the CheckNewNode routine in Alg. 2).

Author's personal copy

Table 7 Branch-and-bound switching cost

Benchmark	NN heuristic		Branch & bound		B&B % reduction relative to NN
	SW cost	% reduction	SW cost	% reduction	
Adpcm	1,207	13.85	1,157	17.42	4.14
Bitcount	1,256	20.2	1,193	24.21	5.02
CRC32	351	18.94	335	22.63	4.56
Dijkstra	1,494	17.82	1,418	22.00	5.09
FFT	1,764	13.53	1,648	19.22	6.58
GSM	30,146	7.21	28,871	11.13	4.23
Jpeg	105,827	9.52	101,025	13.62	4.54
Patricia	1,621	9.64	1,558	13.15	3.89
Qsort	652	15.54	582	24.61	10.74
Rijndael	20,174	11.25	19,294	15.12	4.36
Sha	1,307	7.57	1,215	14.07	7.04
Stringsearch	3,129	8.16	2,989	12.27	4.47
Susan	23,768	12.5	22,762	16.20	4.23
Typeset	495,272	12.76	471,427	16.96	4.81
TOTAL	687,968	11.99	655,474	16.15	4.72

Time limit = 10 ms/instr

6.4 Branch-and-bound versus heuristic performance

The results of the previous section show that the best overall results are achieved using a combination of edge-based lower bounds and history utilization. This combination of pruning techniques is used in this section to evaluate the performance of the proposed branch-and-bound algorithm compared to the NN heuristic.

Table 7 shows the switching cost produced by the branch-and-bound algorithm and the NN heuristic as well as the percentage reduction relative to the CP heuristic for each benchmark. The last column shows the percentage cost reduction achieved by the branch-and-bound algorithm relative to the NN algorithm. It is noted that the difference between the branch-and-bound cost and the NN cost is significant on all benchmarks. Overall, the branch-and-bound algorithm produces a reduction of 16.15 % in switching cost relative to CP, while the NN heuristic produces a reduction of 11.99 % relative to CP. The total switching cost produced by the branch-and-bound algorithm is 4.72 % less than that produced by the NN heuristic. The maximum cost reduction by the branch-and-bound algorithm is seen on the Qsort benchmark, where the branch-and-bound switching cost is 24.61 % lower than the CP switching cost and 10.74 % lower than the NN cost.

6.5 Time limit

Table 8 shows the impact of increasing the time limit. Branch-and-bound statistics are shown for different time limits ranging from 10 ms/instr to 1500 ms/instr. The results

Author's personal copy

Table 8 Impact of increasing the time limit

Stat	10 ms/instr	100 ms/instr	1,000 ms/instr	1,500 ms/instr
BBs timed out	23	7	1	1
BBs timed out and improved	23	7	1	1
BBs timed out and not improved	0	0	0	0
Largest BB solved optimally	598	598	598	598
Largest BB timed out and improved	248	199	125	125
Smallest BB timed out	17	27	125	125
Total switching cost	655,474	655,356	655,340	655,340

Pruning techniques: history utilization and bidirectional edge DLB

in this table were generated using the best combination of pruning techniques (history utilization and bidirectional edge DLBs).

By increasing the time limit from 10 ms/instr to 100 ms/instr, the number of timeouts dropped from 23 to 7, and when increasing the time limit to 1000 ms/instr, only one basic block with size 125 timed out. Further increase in the time limit did not result in any change. Even longer time limits were attempted, and this particular basic block still timed out. Looking at the corresponding improvement in the total switching cost shows that although increasing the time limit from 10 ms/instr to 1,000 ms/instr resulted in solving 16 additional basic blocks to optimality, the corresponding decrease in switching cost is only 0.02 %. From a practical point of view, such a small decrease in cost does not justify using a larger time limit. Therefore, it is concluded that most of the benefit from the branch-and-bound algorithm can be achieved using a small time limit that keeps the solution time within a practically acceptable range.

6.6 Comparison with an MILP-based algorithm

An experiment was conducted to compare the performance of the branch-and-bound algorithm proposed in this article with the MILP-based algorithm proposed by Montemanni et al. [16]. The algorithm of Montemanni et al. was chosen, because it is the most recent MILP-based algorithm for the SOP. The MILP-based algorithm was applied to the hardest 23 instances that our proposed algorithm did not solve within a time limit of 10 ms/instr (see Table 8). For logistic reasons, the comparison could not be carried out in the same lab, but the two algorithms were run on two processors with comparable performance. The MILP-based algorithm was run on an AMD Opteron 250 processor, while the proposed algorithm was run on an Intel Core 2 Duo processor. With a time limit of 2 minutes, the proposed algorithm solved all but three of these instances, while the MILP-based algorithm could only solve two instances. The solution times of the MILP-based algorithm for the two instances that it could solve were an order of magnitude larger than the corresponding solution times taken by the proposed algorithm (5.9 s compared to 0.45 s for the first instance, and 28.1 s compared to 0.98 s for the second instance). For most of the instances that were

Author's personal copy

solved by the proposed algorithm but could not be solved by the MILP-based algorithm, the proposed algorithm's solution times were less than a minute. These results give a strong indication that the proposed algorithm out-performs the MILP-based algorithm on the compiler instances studied in this article.

7 Conclusions and future work

This paper formulates the compiler optimization problem of minimizing the switching activity as a sequential ordering problem and presents a practically efficient branch-and-bound algorithm for solving this problem optimally. The efficiency is achieved by applying a set of pruning techniques that significantly reduce the number of explicitly enumerated solutions. The algorithm was applied to about 30 thousand instances taken from 14 MiBench benchmarks [10] and could optimally solve 99.8 % of these instances with a time limit of 10 ms per node. The branch-and-bound algorithm gave an improvement of 16 % in switching cost relative to a switching-cost-agnostic heuristic and a 5 % improvement relative to the nearest-neighbor heuristic used in previous work.

Future work will consider multiple ideas for improving the proposed algorithm. The algorithm may be improved by computing better upper and lower bounds. Better upper bounds may be computed using more precise heuristics, such as the HAS-SOP heuristic [15], to construct the initial feasible solution. With a tight upper bound, the proposed branch-and-bound algorithm is expected to reach optimality much more quickly.

From a compiler point of view, the algorithm presented in this paper only addresses switching energy and does not account for the schedule length, because it targets modern out-of-order processors that minimize pipeline stalls at run time. Future work may address the more complex problem of balancing schedule length and switching energy on in-order processors that rely on the compiler to minimize the schedule length.

Acknowledgments This research was partially supported by a Google Faculty Research Award granted in August 2013. The authors are extremely grateful to Prof. Roberto Montemanni of the University of Applied Sciences of Southern Switzerland for providing the experimental results reported in Sect. 6.6 for their MILP-based algorithm. The authors thank the Computer Science (CS) Department at PSUT for providing the machine that was used to perform the experimental evaluation. We specially thank Husam Abed and Waseem F. Faous from the system administration team at PSUT for the technical support they provided to us. We also thank the anonymous reviewers for their constructive comments that led to significantly improving the final version of this paper.

References

1. Anghinolfi, D., Montemanni, R., Paolucci, M., Gambardella, L.M.: A particle swarm optimization approach for the sequential ordering problem. Comput. Oper. Res. **38**(7), 1085–2076 (2011)
2. Ascheuer, N., Junger, M., Reinelt, G.: A bracnh & cut algorithm for the asymmetric travelling salesman problem with precedence constraints. Comput. Optim. Appl. **17**(1), 61–84 (2000)
3. Bednarski, A., Kessler, C.: Energy-optimal integrated VLIW code generation. In: Proceedings of the 11th Workshop on Compilers for Parallel Computers (2004)

Author's personal copy

4. Charikar, M., Motwani, R., Raghavan, P., Silverstien, C.: Constrained TSP and low-power computing. In: Proceedings of the Workshop on Algorithms and Data Structures. Lecture Notes in Computer Science, vol. 1272, pp. 104–115. Springer, Berlin (1997)
5. Choi, K., Chatterjee, A.: Efficient instruction-level optimization methodology for low-power embedded systems. In: Proceedings of the International Symposium on System Synthesis (2001)
6. Cooper, K., Torczon, L.: Engineering a Compiler. Morgan Kaufmann, Burlington (2004)
7. Cormen, T., Leiserson, C., Rivest, R., Stein, C.: Introduction to Algorithms, 3rd edn. MIT Press, Cambridge (2009)
8. Dorigo, M., Gambardella, L.M.: Ant colony system: a cooperative learning approach to the traveling salesman problem. IEEE Trans. Evol. Comput. **1**(1), 53–66 (1997)
9. Escudero, L.: An inexact algorithm for the sequential ordering problem. Eur. J. Oper. Res. **37**(2), 236–249 (1988)
10. Guthaus, M.R., Ringenberg, J.S., Ernst, D., Austin, T.M., Mudge, T., Brown, R.B.: MiBench: A free, commercially representative embedded benchmark suite. In: Proceedings of the IEEE International Workshop on Workload Characterization (WWC-4) (2001)
11. Held, M., Karp, R.M.: The traveling salesman problem and minimum spanning trees. Oper. Res. **18**(6), 1138–1162 (1970)
12. Kandemir, M., Vijaykrishnan, N., Irwin, M.J.: Compiler Optimizations for Low Power Systems, Power Aware Computing, vol. 10. Kluwer Academic Publishers, Norwell (2002)
13. Karan, M., Skorin-Kapov, N.: A branch and bound algorithm for the sequential ordering problem. In: Proceedings of the 34th International Convention (MIPRO), pp. 452–457. IEEE (2011)
14. Lee, C., Lee, J.-K., Hwang, T.: Compiler optimization on VLIW instruction scheduling for low power. ACM Trans. Design Autom. Electron. Syst. (TODAES) **8**(2), 252–268 (2003)
15. Gambardella, L.M., Dorigo, M.: HAS-SOP: Hybrid ant system for the sequential ordering problem. Technical Report, Istituto Dalle Molle Di Studi Sull Intelligenza Artificiale (1997)
16. Montemanni, R., Mojana, M., Di Caro, G., Gambardella, L.M.: A decomposition-based exact approach for the sequential ordering problem. J. Appl. Oper. Res. **5**(1), 2–13 (2013)
17. Pallister, J., Hollis, S., Bennett, J.: Identifying compiler options to minimize energy consumption for embedded platforms. Cornell University Library, CS, Performance. arXiv:1303.6485 [cs.PF] (2013)
18. Parikh, A., Kim, S., Kandemir, M., Vijaykrishnan, N., Irwin, M.J.: Instruction scheduling for low power. J. VLSI Signal Process. **37**, 129–149 (2004)
19. Shao, Z., Xiao, B., Xue, C., Zhuge, Q., Sha, E.: Loop scheduling with timing and switching-activity minimization for VLIW DSP. ACM Trans. Design Autom. Electron. Syst. (TODAES) **11**(1), 165–185 (2006)
20. Shobaki, G., Wilken, K.: Optimal superblock scheduling using enumeration. In: Proceedings of the 37th International Symposium on Microarchitecture (2004)
21. Shobaki, G.: Optimal global instruction scheduling using enumeration. PH.D dissertation, Department of Computer Science, UC Davis (2006)
22. Shobaki, G., Wilken, K., Heffernan, M.: Optimal trace scheduling using enumeration. ACM Trans. Arch. Code Optim. (TACO), **5**(4), Article 19 (2009)
23. Shobaki, G., Shawabkeh, M., Abu-Rmaileh, N.: Pre-allocation instruction scheduling with register pressure minimization using a combinatorial optimization approach. ACM Trans. Arch. Code Optim. (TACO), **10**(3), Article 14 (2013)
24. Skiena, S.: The Algorithm Design Manual. Springer, Berlin (2010)
25. Su, C.-L., Tsui, C.Y., Despain, A.M.: Low power architecture design and compilation techniques for high-performance processors. In: Proceedings of the IEEE COMPCON, San Francisco, CA (1994)
26. Tomiyama, H., Ishihara, T., Inoue, A., Hiroto, H.: In: Proceedings of the Conference on Design, Automation and Test in Europe (DATE), pp. 855–860 (1998)
27. Valluri, M., John, L.: Is Compiling for Performance == Compiling for Power?. In: Proceedings 5th Annual Workshop on Interaction between Compilers and Computer Architecture (INTERACT-5), held with HPCA 2001 (2001)
28. Tiwari, V., Malik, S., Wolfe, A.: Compilation Techniques for Low Energy: An Overview. Presented at the Symposium on Low-Power Electronics, San Diego, CA (1994)